

Adding Tests to Untestable Legacy Code



Published on March 19, 2023



Webstack
Builders

Table of Contents

Characterization Tests	3
The Discovery Process	4
Writing Effective Characterization Tests	6
Finding Seams	8
Seam Types	9
Identifying Seams in Legacy Code	12
Breaking Dependencies	14
Extract and Override	15
Parameterize Constructor	17
Introduce Instance Delegator	19
The Strangler Fig Approach	22
Incremental Testing Strategy	22
Golden Master Testing	24
Quick Reference: Common Legacy Code Patterns	27
Static Cling	27
Hidden Dependencies	29
Monster Methods	31
Building the Safety Net	33
Incremental Coverage Strategy	33
Test Organization	34
Conclusion	37



You've inherited a codebase with no tests. The classes are tightly coupled, dependencies are created inline, global state is everywhere, and methods run hundreds of lines. The standard advice—"write unit tests"—assumes the code was designed for testability. It wasn't.

Here's the paradox: you can't refactor safely without tests, but you can't write tests without refactoring. Every attempt to instantiate a class pulls in a database connection, an email client, three external APIs, and a configuration file. A previous developer tried to add tests and gave up after realizing every test required a running database, mail server, and network access to production APIs.

The code is "untestable." Except it isn't.

Consider a real scenario: a team inherits an `OrderProcessor` class. It's 2,000 lines, creates its own database connections, sends emails directly, calls external inventory APIs, and reads from configuration files scattered across the filesystem. No tests exist. The class touches revenue-critical paths, and the team needs to modify it for a new payment provider.

Within two weeks, using the techniques in this article, they have 80% of the critical paths under test **without changing production behavior**. The "untestable" code is now testable. The difference isn't magic — it's specific techniques: characterization tests to capture existing behavior, seam identification to find injection points, and surgical dependency breaking that isolates without altering.

WARNING

The goal isn't 100% coverage — it's getting enough safety net to make changes with confidence. Focus on the code you need to change, not comprehensive coverage of code that works and won't be touched.

Characterization Tests

Traditional unit tests verify that code does what it *should* do — you write a test based on a specification, and the test fails if the code doesn't match. Characterization tests flip this: they capture what the code *actually* does, regardless of intent. You're not testing against a spec; you're documenting observed behavior.



The distinction matters for legacy code. You don't have a spec. The original authors are gone. The code has undocumented edge cases, implicit business rules buried in conditionals, and behaviors that might be bugs or might be features – you can't tell. Characterization tests don't try to answer "is this correct?" They answer "what does this do?" and lock it in.

The Discovery Process

The technique is simple: call the code with an input, observe what it returns, and write a test asserting that exact output. Here's the mental shift:

characterization-vs-unit_spec.rb

```
1  # Traditional unit test: Tests intended behavior based on specification
2  RSpec.describe 'DiscountCalculator' do
3    it 'applies 10% discount for orders over $100' do
4      expect(DiscountCalculator.calculate(150)).to eq(15)
5    end
6  end
7
8  # Characterization test: Tests ACTUAL behavior through observation
9  RSpec.describe 'DiscountCalculator (characterization)' do
10   it 'returns 15 when given 150' do
11     # Observed output; lock it in
12     expect(DiscountCalculator.calculate(150)).to eq(15)
13   end
14
15   it 'returns 0 when given 50' do
16     # Even if this seems wrong, it's what the code does
17     expect(DiscountCalculator.calculate(50)).to eq(0)
18   end
19
20   it 'returns 12 when given 99.99' do
21     # Edge case discovered – might be a bug, but document it
22     expect(DiscountCalculator.calculate(99.99)).to eq(12)
23   end
24 end
```

Characterization tests document observed behavior, not intended behavior.

The process to discover behavior is deliberately low-tech: write a test with an obviously wrong expected value, run it, and let the failure message tell you the actual output. Then update the test with the real value.



behavior_discovery_spec.rb

```
1  RSpec.describe 'LegacyPriceCalculator (discovery)' do
2    let(:calculator) { LegacyPriceCalculator.new }
3
4    it 'discovers behavior for standard order' do
5      order = OrderFactory.create(items: 3, subtotal: 100, customer: 'regular')
6
7      # First run: use obviously wrong expected value
8      # expect(calculator.calculate(order)).to eq(-1)
9      # Test fails: "expected: -1\n      got: 94.50"
10
11     # Now we know the actual output:
12     expect(calculator.calculate(order)).to eq(94.50)
13   end
14
15   it 'discovers behavior for premium customer' do
16     order = OrderFactory.create(items: 3, subtotal: 100, customer: 'premium')
17     # Discovered: premium customers get different pricing
18     expect(calculator.calculate(order)).to eq(85.00)
19   end
20
21   it 'discovers edge case - zero items' do
22     order = OrderFactory.create(items: 0, subtotal: 0, customer: 'regular')
23     # Discovered: returns minimum fee even for empty orders (bug or feature?)
24     expect(calculator.calculate(order)).to eq(5.00)
25   end
26 end
```

Using failing tests to discover actual behavior.

That last test – the \$5.00 minimum fee for empty orders – is exactly the kind of thing characterization tests reveal. Is it a bug? Intentional? You don't know, and for now, you don't care. You're building a safety net, not making judgments. Document it with a comment, move on, and address it later when you have coverage.

Writing Effective Characterization Tests

Comprehensive characterization requires systematic exploration. Three strategies help ensure you're covering the behavior that matters:



Input partitioning divides inputs into equivalence classes. For an order processor, that might be order sizes (empty, single, small, bulk), customer types (guest, regular, premium), payment methods, and shipping regions. Test at least one representative from each partition, and key combinations:

input_partitioning_spec.rb

```
1  RSpec.describe 'OrderProcessor (input partitioning)' do
2    let(:processor) { OrderProcessor.new }
3
4    [
5      {
6        size: 'single',
7        customer: 'regular',
8        payment: 'credit',
9        region: 'domestic'
10     },
11     {
12       size: 'bulk',
13       customer: 'enterprise',
14       payment: 'invoice',
15       region: 'international'
16     },
17     {
18       size: 'empty',
19       customer: 'guest',
20       payment: 'prepaid',
21       region: 'restricted'
22     }
23   ].each do |params|
24     it "processes order: " \
25       "#{params[:size]}, #{params[:customer]}, " \
26       "#{params[:payment]}, #{params[:region]}" do
27       order = OrderFactory.create(params)
28       result = processor.process(order)
29       expect(result).to match_snapshot("order_#{params[:size]}_" \
30         "#{params[:customer]}_#{params[:payment]}_#{params[:region]}")
31     end
32   end
33 end
34 end
```



Testing across input partitions with snapshots.

Boundary value analysis focuses on thresholds – the places where behavior changes. If there's a discount tier at \$100, test \$99.99, \$100.00, and \$100.01. Legacy code often has off-by-one errors or inconsistent boundary handling that you'll only catch by probing the edges:

boundary_testing_spec.rb

```
1  RSpec.describe 'DiscountCalculator (boundaries)' do
2    let(:calculator) { DiscountCalculator.new }
3
4    [
5      [99.99, 0],      # Just under threshold
6      [100.00, 10],    # At threshold
7      [100.01, 10.001], # Just over threshold
8      [999.99, 99.999], # Near next tier
9      [1000.00, 150]   # At next tier
10   ].each do |amount, expected|
11     it "calculates discount for $#{amount} as #{expected}" do
12       result = calculator.get_discount(amount)
13       expect(result).to be_within(0.01).of(expected)
14     end
15   end
16 end
```

Probing boundary values to discover threshold behavior.

Error condition exploration tests what happens when inputs are invalid. Don't assume the code throws – observe what it actually does. Legacy code often has surprising error handling:

error_exploration_spec.rb

```
1  RSpec.describe 'PaymentProcessor (error conditions)' do
2    let(:processor) { PaymentProcessor.new }
3
4    it 'handles null payment method' do
5      result = processor.process(amount: 100, method: nil)
6      # Discovered: defaults to cash instead of throwing
7      expect(result.method).to eq('cash')
8      expect(result.status).to eq('pending_verification')
```



```
9     end
10
11     it 'handles negative amounts' do
12       result = processor.process(amount: -50, method: 'credit')
13       # Discovered: processes as refund instead of rejecting
14       expect(result.type).to eq('refund')
15       expect(result.amount).to eq(50)
16     end
17   end
```

Exploring error handling without assumptions.

For complex outputs – reports, invoices, nested data structures – snapshot testing captures the entire structure without you having to write assertions for every field. When the snapshot changes, the test fails and shows you a diff. You review whether the change was intentional.

INFO

Characterization tests are not about correctness – they’re about documenting current behavior. If you discover a bug during characterization, document it with a comment but don’t “fix” the test. Fix the bug later, after you have your safety net in place.

Finding Seams

Michael Feathers introduced the concept of seams in **Working Effectively with Legacy Code**, and it remains the most useful mental model for making untestable code testable. A seam is a place where you can alter program behavior without editing the code at that location. The seam itself doesn’t change – you change behavior at what Feathers calls the “enabling point.”

The distinction matters because legacy code often can’t be edited safely. You don’t have tests, so any edit risks breaking something. Seams let you substitute behavior for testing purposes without touching the production logic you’re trying to protect.

Consider a method that sends emails. The email-sending code is deep inside a 500-line method that also processes orders, updates inventory, and logs analytics. You can’t easily extract the email logic – too risky without tests. But if you can find a seam, you can replace the email sender with a test double that captures



what **would** have been sent, without changing the method itself.

Seam Types

seam_types.rb

```
1  # Seams: Places where you can alter behavior without editing code
2  # Michael Feathers: "A place where you can alter behavior in your program without
   editing in that place"
3
4  # Type 1: Object Seam (most common in OOP)
5  class OrderService
6    def initialize(email_service = EmailService.new)
7      # Object seam - can inject test double
8      @email_service = email_service
9    end
10 end
11
12 # Type 2: Link Seam (module/import level)
13 # production.rb
14 require_relative 'email_service'
15
16 # test.rb
17 RSpec.describe 'OrderService link seam' do
18   before do
19     stub_const('EmailService', Class.new do
20       def send_email(*); { success: true }; end
21     end)
22   end
23
24   it 'mocks email service' do
25     service = OrderService.new(EmailService.new)
26     expect(service.instance_variable_get(:@email_service).send_email).to eq({ success:
true })
27   end
28 end
29
30 # Type 3: Preprocessor Seam (compile-time, less common in Ruby)
31 # Ruby doesn't have a preprocessor, but you can use environment flags
32 def calculate_value
33   if ENV['TESTING']
34     :mock_value
```



```
35     else
36       real_calculation
37     end
38   end
39
40   # Type 4: Subclass Seam (override in test subclass)
41   class PaymentProcessor
42     def call_payment_gateway(amount)
43       # Real implementation calls external API
44       ExternalGateway.charge(amount)
45     end
46
47     def process(order)
48       payment = call_payment_gateway(order.total)
49       # ... rest of logic
50     end
51   end
52
53   # In tests:
54   class TestablePaymentProcessor < PaymentProcessor
55     def call_payment_gateway(amount)
56       # Override to avoid external call
57       { success: true, transaction_id: 'test-123' }
58     end
59   end
60
61   RSpec.describe TestablePaymentProcessor do
62     it 'returns test payment result' do
63       processor = TestablePaymentProcessor.new
64       result = processor.call_payment_gateway(100)
65       expect(result[:success]).to be true
66       expect(result[:transaction_id]).to eq('test-123')
67     end
68   end
```

Types of seams.

The **object seam** is the most common and usually the cleanest. You pass a dependency through a constructor or method parameter, and the enabling point is the call site where you can pass a different implementation. Ruby makes this trivially easy with default arguments – existing callers get production behavior, tests pass doubles. The same pattern works across many different frameworks.



Language/Framework	Object Seam Technique
Ruby	Default arguments in initialize
C# / Java	Constructor overloads or optional parameters
PHP	Default parameter values
TypeScript	Optional parameters with nullish coalescing (??)
Spring Boot	@Autowired with test @Configuration bindings
Laravel	Service container with <code>\$this->app->bind()</code> in tests
.NET	IServiceCollection with test service registration

Object seam techniques by language and framework.

Link seams operate at the module level. In Ruby, you can use `stub_const` to replace a class entirely. In TypeScript/JavaScript, Jest's `jest.mock()` intercepts imports. PHP's Mockery can override class instantiation. Java has PowerMock for static method mocking, though it's a code smell. The enabling point is the test setup. Link seams are powerful but fragile – they couple tests to implementation details like class names, and they break if someone renames things.

Subclass seams work by extracting behavior into a protected method, then overriding it in a test subclass. The enabling point is inheritance. This technique is underrated for legacy code because it requires minimal changes to the original class – you extract one line into a method, and suddenly you have a seam.

The preprocessor seam is mostly a C/C++ concept, but the pattern applies anywhere you use environment-based branching. Rails' `Rails.env.test?`, Laravel's `app()->environment('testing')`, Spring's `@Profile("test")`, and Node's `process.env.NODE_ENV === 'test'` are all effectively preprocessor seams. Use them sparingly – they litter production code with test concerns.

Identifying Seams in Legacy Code

When you're staring at a tightly coupled class, finding seams requires a systematic scan. Look for object instantiation with `new` or `.new` –each one is a potential object seam waiting to be parameterized. Look for class method calls like `SomeClass.do_thing` –each one could become an instance delegator. Look for



global access patterns: singletons, environment variables, file reads, network calls.

seam_identification.rb

```
1  # Finding seams in untestable legacy code
2
3  # BEFORE: Tightly coupled, no seams
4  class LegacyOrderProcessor
5    def process_order(order_id)
6      # Direct database call - no seam
7      db = DatabaseConnection.new(CONFIG[:connection_string])
8      order = db.query("SELECT * FROM orders WHERE id = '#{order_id}'")
9
10     # Direct file system access - no seam
11     config = JSON.parse(File.read('/etc/app/config.json'))
12
13     # Direct external API call - no seam
14     inventory = InventoryApi.check(order[:sku])
15
16     # Direct email sending - no seam
17     mailer = StpMailer.new(config['smtp'])
18     mailer.send(order[:customer_email], 'Order Confirmation', template)
19
20     # Direct logging - no seam (usually acceptable)
21     Rails.logger.info("Processed order #{order_id}")
22   end
23 end
24
25 # ANALYSIS: Finding potential seams
26 #
27 # 1. DatabaseConnection instantiation
28 #   - Could extract to factory method (subclass seam)
29 #   - Could add constructor parameter (object seam)
30 #
31 # 2. File system read
32 #   - Could extract to separate method (subclass seam)
33 #   - Could inject config object (object seam)
34 #
35 # 3. InventoryApi call
36 #   - Could inject API client (object seam)
37 #   - Could mock InventoryApi (link seam)
38 #
```



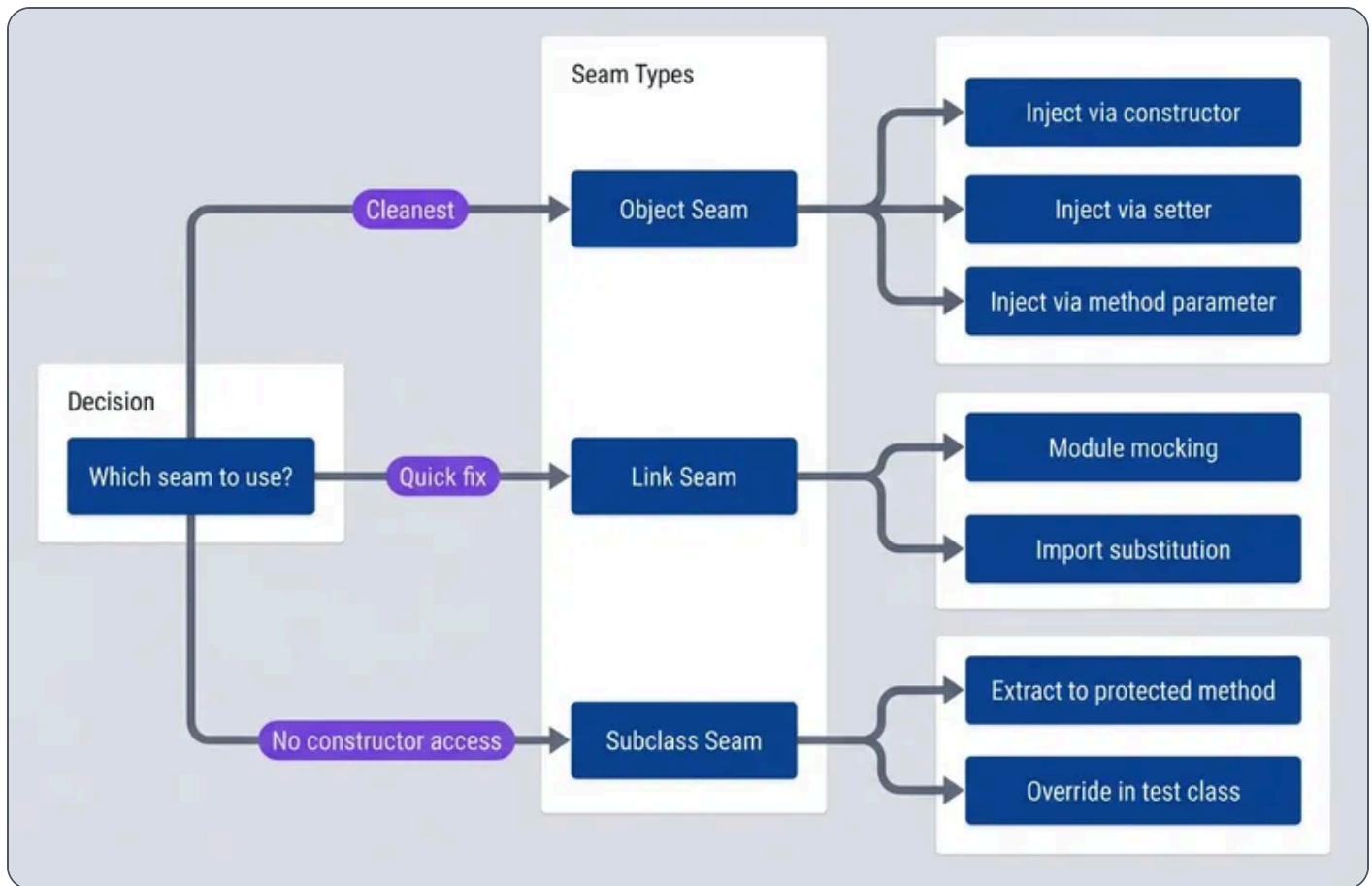
```
39 # 4. Smtplib instantiation
40 #   - Could inject mailer interface (object seam)
41 #   - Could extract to protected method (subclass seam)
42
43 # PATTERN: Introduce Sensing Variable (temporary seam)
44 class LegacyOrderProcessor
45   # Temporary field for testing - remove after refactoring
46   attr_accessor :testing_last_email_sent
47
48   def process_order(order_id)
49     # ... existing code ...
50
51     mailer.send(order[:customer_email], 'Order Confirmation', template)
52
53     # Sensing: capture what was sent for test verification
54     self.testing_last_email_sent = {
55       to: order[:customer_email],
56       subject: 'Order Confirmation'
57     }
58   end
59 end
```

Identifying seams in legacy code.

The sensing variable pattern deserves special attention. Sometimes you can't easily intercept a dependency, but you *can* add a field that captures what happened. It's ugly – test-specific code in production – but it's a temporary scaffold. Add the sensing variable, write your characterization tests, then refactor toward proper seams and remove the sensing code. The tests survive because they now use better seams.

When choosing which seam type to use, prefer object seams for long-term maintainability. They make dependencies explicit and support proper dependency injection. But when you need tests *now* and can't change constructor signatures (maybe there are 50 call sites), link seams or subclass seams get you there faster. You can always refactor later – once you have tests.





Seam type decision.

✓ SUCCESS

Look for the `new` keyword and class method calls – these are often where seams are missing. Every `.new` inside a method is a dependency that's hard to test. Every `SomeClass.method` call is a hidden dependency.

Breaking Dependencies

Once you've identified seams, you need techniques to exploit them. The goal is surgical: make the minimum change that enables testing, without altering the behavior you're trying to protect. These techniques work across languages and frameworks – the syntax differs, but the patterns are universal.



Extract and Override

Extract and Override is the fastest path to testability when you can't change a class's constructor. You take the problematic code – the `new Database()`, the `fetch()` call, the `new Date()` – and extract it into a protected method. The original method calls the new method instead of the dependency directly. Then in your test, you subclass and override the protected method to return controlled values.

The beauty is that production code doesn't change behavior at all. The protected method still calls the real dependency. Only in tests do you swap it out. This technique works in any language with inheritance: Java, C#, PHP, TypeScript, Ruby, Python.

extract_and_override_spec.rb

```
1  # Technique: Extract and Override (Subclass Seam)
2
3  # BEFORE: Untestable - creates its own dependencies
4  class InvoiceGenerator
5    def generate(customer_id)
6      # Hard-coded database dependency
7      db = Database.new(ENV['DB_CONNECTION'])
8      customer = db.find_customer(customer_id)
9
10     # Hard-coded external API call
11     tax_rate = TaxApi.fetch_rate(customer.state)
12
13     # Hard-coded date (makes tests non-deterministic)
14     invoice_date = Time.now
15
16     Invoice.new(customer, tax_rate, invoice_date)
17   end
18 end
19
20 # AFTER: Extract to protected methods (minimal change)
21 class InvoiceGenerator
22   def generate(customer_id)
23     customer = get_customer(customer_id)
24     tax_rate = get_tax_rate(customer.state)
25     invoice_date = get_current_date
26
27     Invoice.new(customer, tax_rate, invoice_date)
28   end
29 end
```



```
30     protected
31
32     def get_customer(customer_id)
33       db = Database.new(ENV['DB_CONNECTION'])
34       db.find_customer(customer_id)
35     end
36
37     def get_tax_rate(state)
38       TaxApi.fetch_rate(state)
39     end
40
41     def get_current_date
42       Time.now
43     end
44   end
45
46   # TEST: Override protected methods in a test subclass
47   class TestableInvoiceGenerator < InvoiceGenerator
48     def initialize(customer:, tax_rate:, date:)
49       @test_customer = customer
50       @test_tax_rate = tax_rate
51       @test_date = date
52     end
53
54     protected
55
56     def get_customer(_customer_id)
57       @test_customer
58     end
59
60     def get_tax_rate(_state)
61       @test_tax_rate
62     end
63
64     def get_current_date
65       @test_date
66     end
67   end
68
69   # Now we can test the core logic without dependencies
70   RSpec.describe InvoiceGenerator do
71     it 'generates invoice with correct total and date' do
72       customer = double('Customer', id: 123, state: 'CA', balance: 100)
```




```
73     generator = TestableInvoiceGenerator.new(  
74         customer: customer,  
75         tax_rate: 0.0825,  
76         date: Time.new(2024, 1, 15)  
77     )  
78  
79     invoice = generator.generate(123)  
80  
81     expect(invoice.total).to eq(108.25)  
82     expect(invoice.date).to eq(Time.new(2024, 1, 15))  
83 end  
84 end
```

Extract and override technique.

The test subclass feels awkward if you're used to mocking frameworks, but it has advantages: no magic, no reflection, obvious what's happening. The downside is proliferating test subclasses – one per class you're testing. It's a stepping stone. Once you have tests, refactor toward constructor injection.

Parameterize Constructor

Parameterize Constructor is the classic dependency injection pattern. Instead of creating dependencies internally, the class receives them through its constructor. The key insight for legacy code: use optional parameters with defaults. Existing call sites don't need to change – they get production dependencies automatically. Only tests pass explicit doubles.

This pattern is the foundation of modern DI (Dependency Injection) frameworks. Spring Boot's `@Autowired`, Laravel's service container, .NET's `IServiceCollection`, and NestJS's providers all build on this concept. Even without a framework, manual constructor injection works everywhere.

parameterize_constructor_spec.rb

```
1  # Technique: Parameterize Constructor (Object Seam)  
2  
3  # BEFORE: Dependencies created internally  
4  class NotificationService  
5      def initialize  
6          @mailer = EmailClient.new(ENV['SMTP_CONFIG'])  
7          @sms = SmsClient.new(ENV['TWILIO_CONFIG'])
```



```
8     @logger = FileLogger.new('/var/log/notifications.log')
9   end
10
11   def notify(user, message)
12     @logger.log("Notifying \\#{user.id}")
13
14     @mailer.send(user.email, message) if user.preferences[:email]
15     @sms.send(user.phone, message) if user.preferences[:sms]
16   end
17 end
18
19 # AFTER: Dependencies injectable with defaults (preserves existing behavior)
20 class NotificationService
21   def initialize(mailer: nil, sms: nil, logger: nil)
22     @mailer = mailer || EmailClient.new(ENV['SMTP_CONFIG'])
23     @sms = sms || SmsClient.new(ENV['TWILIO_CONFIG'])
24     @logger = logger || FileLogger.new('/var/log/notifications.log')
25   end
26
27   def notify(user, message)
28     @logger.log("Notifying \\#{user.id}")
29
30     @mailer.send(user.email, message) if user.preferences[:email]
31     @sms.send(user.phone, message) if user.preferences[:sms]
32   end
33 end
34
35 # TEST: Inject test doubles
36 RSpec.describe NotificationService do
37   let(:mock_mailer) { double('EmailClient', send: true) }
38   let(:mock_sms) { double('SmsClient', send: true) }
39   let(:mock_logger) { double('FileLogger', log: true) }
40   let(:user) do
41     OpenStruct.new(
42       id: 1,
43       email: 'test@example.com',
44       phone: '555-1234',
45       preferences: { email: true, sms: false }
46     )
47   end
48
49   it 'sends email when user prefers email' do
50     service = NotificationService.new(
```



```
51     mailer: mock_mailer,  
52     sms: mock_sms,  
53     logger: mock_logger  
54   )  
55   service.notify(user, 'Hello')  
56   expect(mock_mailer).to have_received(:send).with('test@example.com', 'Hello')  
57   expect(mock_sms).not_to have_received(:send)  
58 end  
59 end
```

Parameterize constructor technique.

Notice the defaults preserve backward compatibility. Every existing `new NotificationService()` call still works – it gets production dependencies. This is crucial in legacy codebases where you might have dozens of call sites. You don't want to update them all before you can write your first test.

Introduce Instance Delegator

Instance Delegator handles the special case of static method calls – what Feathers calls “static cling.” Static methods can't be overridden through inheritance, and many mocking frameworks struggle with them. The solution: wrap the static call in an instance method, then use Extract and Override or constructor injection on that wrapper.

In Java, this often means wrapping calls to utility classes or legacy static factories. In PHP, it's facade calls. In Ruby, it's class methods. In C#, it's static helpers. The pattern is the same: create an instance method that delegates to the static, then test against the instance method.

instance_delegator.rb

```
1  # Technique: Introduce Instance Delegator (for static methods)  
2  
3  # BEFORE: Static method call - hard to test  
4  class OrderValidator  
5    def validate(order)  
6      # Static utility call - can't override or mock easily  
7      is_valid_address = AddressValidator.validate(order.shipping_address)  
8      is_valid_payment = PaymentValidator.validate(order.payment_method)  
9    end  
10  end
```



```

10     return { valid: false, reason: 'Invalid address' } unless is_valid_address
11     return { valid: false, reason: 'Invalid payment method' } unless is_valid_payment
12     { valid: true }
13 end
14 end
15
16 # AFTER: Wrap static calls in instance methods
17 class OrderValidator
18   def validate(order)
19     is_valid_address = validate_address(order.shipping_address)
20     is_valid_payment = validate_payment(order.payment_method)
21
22     return { valid: false, reason: 'Invalid address' } unless is_valid_address
23     return { valid: false, reason: 'Invalid payment method' } unless is_valid_payment
24     { valid: true }
25   end
26
27   # Instance delegators to static methods
28   protected
29   def validate_address(address)
30     AddressValidator.validate(address)
31   end
32
33   def validate_payment(payment)
34     PaymentValidator.validate(payment)
35   end
36 end
37
38 # Alternative: Extract interface and inject
39 class OrderValidator
40   def initialize(
41     address_validator = AddressValidator,
42     payment_validator = PaymentValidator
43   )
44     @address_validator = address_validator
45     @payment_validator = payment_validator
46   end
47
48   def validate(order)
49     is_valid_address = @address_validator.validate(order.shipping_address)
50     is_valid_payment = @payment_validator.validate(order.payment_method)
51     # ...
52   end

```



53 end

Instance delegator technique.

The alternative approach – injecting the validator classes themselves rather than wrapping their calls – produces cleaner code long-term. You can pass mock classes in tests that respond to `.validate` however you need. This is effectively Parameterize Constructor applied to class method dependencies.

#	Technique	When to Use	Pros	Cons
1	Extract and Override	Quick wins, no API change	Minimal code change	Test class inheritance
2	Parameterize Constructor	Long-term solution	Clean DI pattern	May require interface
3	Instance Delegator	Static method calls	Enables testing	Adds indirection
4	Link Seam (mocking)	Can't modify code	No production changes	Fragile to refactoring

Dependency breaking techniques.

Choose based on your constraints. If you need tests today and can't change the constructor signature, Extract and Override gets you there. If you're building for the long term and can modify call sites, Parameterize Constructor produces better designs. Instance Delegator is your escape hatch for static cling. Link seams are the last resort when you truly can't touch the code.

WARNING

These techniques are stepping stones, not final designs. Extract and Override is faster but creates inheritance-based testing. After you have tests, refactor toward proper dependency injection.



The techniques above work well for individual classes with a few dependencies. But what happens when the problem isn't a class – it's an entire subsystem?

The Strangler Fig Approach

The dependency-breaking techniques get you tests around individual classes. But what about the 3,000-line monolith that does everything? You can't test it as a unit – it *is* the unit. The Strangler Fig pattern, named after the tropical vines that gradually envelop and replace their host trees, offers a path forward.

The idea: wrap the legacy system with a facade that has a clean interface. Write characterization tests against the facade. Then, one responsibility at a time, extract logic from the legacy system into new, tested components. The facade orchestrates both old and new code. Eventually, the legacy system is hollow – all real work happens in the new components – and you remove it entirely.

This pattern works at any scale: a single class, a module, an entire service. It's how you migrate from a legacy monolith to microservices without a big-bang rewrite. It's how you replace a legacy ORM with a new one. The key is incremental progress with continuous delivery – every step is deployable.

Incremental Testing Strategy

strangler_fig.rb

```
1  # Strangler Fig: Gradually wrap and replace legacy code
2
3  # BEFORE: Monolithic untestable class
4  class LegacyOrderSystem
5    # 3000 lines of code
6    # Handles: validation, pricing, inventory, shipping, notifications
7    def process_order(order_data)
8      # ... massive method with everything intertwined
9    end
10 end
11
12 # PHASE 1: Wrap with facade, characterize at boundary
13 class OrderSystemFacade
14   def initialize(legacy = LegacyOrderSystem.new)
15     @legacy = legacy
16   end
17
```



```

18     def process_order(order_data)
19       legacy_input = to_legacy_format(order_data)
20       legacy_result = @legacy.process_order(legacy_input)
21       from_legacy_format(legacy_result)
22     end
23
24     private
25     def to_legacy_format(order_data)
26       # ... convert to legacy format ...
27     end
28
29     def from_legacy_format(legacy_result)
30       # ... convert from legacy format ...
31     end
32   end
33
34   # Characterize the facade
35   RSpec.describe OrderSystemFacade do
36     let(:facade) { OrderSystemFacade.new }
37
38     it 'processes standard order' do
39       input = { items: [], customer: 'test' }
40       result = facade.process_order(input)
41       expect(result).to match_snapshot('standard_order')
42     end
43   end
44
45   # PHASE 2: Extract one responsibility with tests
46   class PricingService
47     def calculate_price(items, customer)
48       # Clean, tested code
49     end
50   end
51
52   class OrderSystemFacade
53     def initialize(legacy = LegacyOrderSystem.new, pricing = PricingService.new)
54       @legacy = legacy
55       @pricing = pricing
56     end
57
58     def process_order(order_data)
59       price = @pricing.calculate_price(order_data[:items], order_data[:customer])
60       legacy_input = to_legacy_format(order_data.merge(calculated_price: price))

```



```
61     legacy_result = @legacy.process_order(legacy_input)
62     from_legacy_format(legacy_result)
63   end
64 end
65
66 # PHASE 3: Continue extracting until legacy is hollow
67 class OrderSystemFacade
68   def initialize(validation:, pricing:, inventory:, shipping:, notifications:)
69     @validation = validation
70     @pricing = pricing
71     @inventory = inventory
72     @shipping = shipping
73     @notifications = notifications
74   end
75
76   def process_order(order_data)
77     validated = @validation.validate(order_data)
78     priced = @pricing.calculate(validated)
79     reserved = @inventory.reserve(priced)
80     shipped = @shipping.schedule(reserved)
81     @notifications.send(shipped)
82     shipped
83   end
84 end
85
86 # PHASE 4: Remove facade, legacy code is gone
```

Strangler fig migration pattern.

Golden Master Testing

Some legacy systems produce outputs so complex that writing assertions for each scenario is impractical. A reporting engine that generates 50-page PDFs. A pricing system with thousands of SKU combinations. An export process that produces elaborate XML documents. For these, golden master testing offers a pragmatic solution.

The technique: run the system against a representative sample of inputs and capture the outputs. This becomes your “golden master”—the canonical record of what the system does. Future test runs compare against this baseline. Any difference triggers investigation: did we change something intentionally, or did we break something?



Golden master tests trade precision for coverage. You're not asserting *why* the output is correct – you're asserting that it hasn't changed. This makes them perfect for legacy systems where understanding every nuance would take months. They're also framework-agnostic: any language can serialize outputs and compare them later.

golden_master_spec.rb

```
1  # Golden Master: Capture comprehensive output for complex systems
2
3  require 'json'
4  require 'time'
5
6  class GoldenMaster
7    GOLDEN_PATH = 'spec/golden_masters/order_processing.json'
8
9    def self.generate(system, samples)
10     results = samples.map do |test_case|
11       {
12         id: test_case[:id],
13         input: test_case[:input],
14         output: system.process(test_case[:input]),
15         timestamp: Time.now.utc.iso8601
16       }
17     end
18     File.write(GOLDEN_PATH, JSON.pretty_generate(results))
19   end
20
21   def self.verify(system)
22     golden = JSON.parse(File.read(GOLDEN_PATH), symbolize_names: true)
23     failures = []
24
25     golden.each do |record|
26       actual = system.process(record[:input])
27       unless actual == record[:output]
28         failures << {
29           id: record[:id],
30           expected: record[:output],
31           actual: actual,
32           diff: actual.to_s != record[:output].to_s ? [record[:output], actual] : nil
33         }
34       end
35     end
36   end
37 end
```



```
37     {
38       total: golden.size,
39       passed: golden.size - failures.size,
40       failed: failures.size,
41       failures: failures
42     }
43   end
44 end
45
46 # Usage in CI
47 RSpec.describe 'Legacy System - Golden Master' do
48   let(:system) { LegacySystem.new }
49
50   it 'matches golden master output' do
51     results = GoldenMaster.verify(system)
52     if results[:failed] > 0
53       puts "Failures: #{results[:failures].to_json}"
54     end
55     expect(results[:failed]).to eq(0)
56   end
57 end
58
59 # When to update golden master:
60 # 1. Intentional behavior change
61 # 2. Bug fix (behavior was wrong)
62 # 3. New feature (new outputs expected)
63 # NEVER update just because tests fail – investigate first!
```

Golden master testing.

✓ SUCCESS

Golden master tests are particularly powerful for legacy systems with complex outputs. They capture “what the system does” across hundreds of scenarios without you needing to understand each one. When they fail, you know something changed – then you investigate whether it was intentional.



Quick Reference: Common Legacy Code Patterns

The techniques covered so far — characterization tests, seams, dependency breaking, strangler fig — are the foundations. This section provides a quick reference for recognizing and addressing the most common anti-patterns you'll encounter. Each pattern maps directly to techniques already discussed; this is a condensed lookup table for when you're staring at problematic code and need to identify your approach quickly.

Static Cling

Static method calls look innocent but create testing nightmares. When your class calls

`DateUtils.getCurrentTime()` or `Database.query()`, you can't substitute test doubles. The dependency is baked into the bytecode, not passed through a seam you control.

The problem exists across all statically-typed languages — Java's static methods, C#'s static classes, PHP's static functions. Even dynamically-typed languages suffer when code calls library functions directly rather than through injectable collaborators.

Two solutions dominate. First, wrap static calls in instance methods, then override in tests (Extract and Override). Second, wrap static access behind an interface and inject it (Adapter pattern). The latter produces better design; the former requires less immediate change.

static_cling.rb

```
1  # Problem: Static method calls that can't be overridden
2
3  # BEFORE: Direct static calls everywhere
4  class ReportGenerator
5    def generate
6      data = DatabaseHelper.query('SELECT * FROM sales')
7      formatted = StringUtils.format_currency(data[:total])
8      timestamp = DateUtils.current_timestamp
9      Report.new(data, formatted, timestamp)
10   end
11 end
12
13 # SOLUTION 1: Wrap in instance methods (Extract and Override)
14 class ReportGenerator
15   def generate
```



```
16     data = query_database('SELECT * FROM sales')
17     formatted = format_currency(data[:total])
18     timestamp = get_timestamp
19     Report.new(data, formatted, timestamp)
20 end
21
22 protected
23
24 def query_database(sql)
25     DatabaseHelper.query(sql)
26 end
27
28 def format_currency(amount)
29     StringUtils.format_currency(amount)
30 end
31
32 def get_timestamp
33     DateUtils.current_timestamp
34 end
35 end
36
37 # SOLUTION 2: Use dependency injection (adapter pattern)
38 class DatabaseAdapter
39     def query(sql)
40         DatabaseHelper.query(sql)
41     end
42 end
43
44 class ReportGenerator
45     def initialize(db = DatabaseAdapter.new)
46         @db = db
47     end
48
49     def generate
50         data = @db.query('SELECT * FROM sales')
51         # ...
52     end
53 end
```

Handling static cling.



Hidden Dependencies

Singletons and global state create invisible coupling. The code *looks* like it has no dependencies – the constructor takes no arguments, the method signature is clean. But inside, it reaches into global state:

`Configuration.instance` , `Logger.global` , `ServiceLocator.resolve()` . You can't see the dependencies from the API, but they're there, and they'll sabotage your tests.

Every framework has its version. Java has the classic Singleton pattern. PHP has facades that hide service container lookups. Node.js has module-level singletons. Rails has `Rails.application.config` . The symptom is always the same: tests pass in isolation but fail when run together, because they're sharing state through these hidden channels.

The cure is exposure: make hidden dependencies visible in the constructor. Keep the defaults pointing to production implementations so existing code doesn't break. Now tests can inject alternatives, and the dependency graph becomes explicit.

hidden_dependencies.rb

```
1  # Problem: Dependencies accessed via global state or singletons
2
3  # BEFORE: Hidden singleton access
4  class OrderProcessor
5    def process(order)
6      # Hidden dependency: singleton access
7      config = Configuration.instance
8      logger = Logger.instance
9      cache = CacheManager.instance
10
11     logger.info("Processing order \\#{order.id}")
12
13     if config.get('feature.newPricing')
14       # ... new pricing logic
15     end
16
17     cache.set("order:#{order.id}", order)
18   end
19 end
20
21 # SOLUTION: Expose dependencies, provide defaults
22 class OrderProcessor
```



```
23     def initialize(config: Configuration.instance, logger: Logger.instance, cache:
CacheManager.instance)
24         @config = config
25         @logger = logger
26         @cache = cache
27     end
28
29     def process(order)
30         @logger.info("Processing order \\#{order.id}")
31
32         if @config.get('feature.newPricing')
33             # ... new pricing logic
34         end
35
36         @cache.set("order:#{order.id}", order)
37     end
38 end
39
40 # ALTERNATIVE: Reset singletons in tests (last resort)
41 RSpec.describe OrderProcessor do
42     before do
43         # Reset singleton state
44         Configuration.reset_instance
45         Logger.reset_instance
46         CacheManager.reset_instance
47
48         # Configure for testing
49         Configuration.instance.set('feature.newPricing', true)
50     end
51
52     after do
53         # Restore production state
54         Configuration.reset_instance
55     end
56 end
```

Handling hidden dependencies.



Monster Methods

Every legacy codebase has them: methods that span hundreds of lines, doing everything from validation to database access to email sending. They're impossible to test as units because they *aren't* units – they're entire subsystems crammed into a single method signature.

Don't try to test monster methods directly. Don't try to refactor them all at once. Instead, use “sprout” techniques to grow tested code alongside the monster – this is the Strangler Fig philosophy applied at method level. Need to add a feature? Write it in a new, tested method, then call it from the monster. Need to fix a bug? Extract that logic into a tested class, then delegate to it.

Over time, the monster becomes a thin shell that orchestrates calls to well-tested components. You never had to understand the monster's full complexity. You never had to write tests for code that might have subtle bugs you'd be enshrining.

monster_methods.rb

```
1  # Problem: Methods too long to understand or test
2
3  # BEFORE: 500-line method
4  class InvoiceProcessor
5      def process_monthly_invoices
6          # Lines 1-100: Load customer data
7          # Lines 101-200: Calculate usage
8          # Lines 201-300: Apply discounts
9          # Lines 301-400: Generate PDF
10         # Lines 401-500: Send emails
11     end
12 end
13
14 # SOLUTION: Sprout Method - add new functionality in tested methods
15 class InvoiceProcessor
16     def process_monthly_invoices(invoices)
17         # ... existing 500 lines unchanged ...
18
19         # NEW: Add late fee calculation (tested separately)
20         late_fees = calculate_late_fees(invoices)
21         apply_late_fees(invoices, late_fees)
22     end
23
```



```

24   # New method is fully tested
25   def calculate_late_fees(invoices)
26     fees = {}
27     invoices.each do |invoice|
28       if overdue?(invoice)
29         fees[invoice.id] = invoice.amount * 0.015
30       end
31     end
32     fees
33   end
34
35   private
36
37   def overdue?(invoice)
38     invoice.due_date < Date.today
39   end
40 end
41
42 # SOLUTION: Sprout Class - extract to new tested class
43 class LateFeeCalculator
44   def calculate(invoices)
45     fees = {}
46     invoices.each do |invoice|
47       if invoice.due_date < Date.today
48         fees[invoice.id] = invoice.amount * 0.015
49       end
50     end
51     fees
52   end
53 end
54
55 class InvoiceProcessor
56   def initialize(late_fee_calculator = LateFeeCalculator.new)
57     @late_fee_calculator = late_fee_calculator
58   end
59
60   def process_monthly_invoices(invoices)
61     # ... existing code ...
62     late_fees = @late_fee_calculator.calculate(invoices)
63     # ...
64   end
65 end

```



Handling monster methods.

INFO

“Sprout” techniques (sprout method, sprout class) let you add tested code without changing existing logic. The monster method stays untouched, but new functionality is clean and tested. Over time, you sprout enough that the monster becomes hollow.

Building the Safety Net

With techniques in hand, you need strategy. Legacy codebases are too large to test comprehensively. You need to focus effort where it matters: code that’s changing, code that’s failing, code that’s critical. Testing stable legacy code that works and won’t be touched is waste.

Think of coverage as insurance. You buy more insurance for valuable, at-risk assets. A module you’re rewriting next month needs heavy coverage. A module that hasn’t changed in three years and has no planned changes needs minimal coverage – maybe just characterization tests to catch accidental modifications.

Incremental Coverage Strategy

Prioritize based on three factors: change frequency (code that changes often needs more coverage), bug history (code that breaks often needs more coverage), and planned work (code you’ll modify soon needs coverage *before* you touch it). Stable, boring code that just works? Leave it alone.

This isn’t about reaching arbitrary coverage numbers. It’s about building confidence where you need it. 80% coverage on code you’re actively refactoring is more valuable than 20% coverage spread across the entire codebase.

coverage_strategy.rb

```
1  # Strategy: Prioritize coverage by change risk
2
3  CoverageTarget = Struct.new(:file, :change_frequency, :bug_history, :planned_changes,
4                               :current_coverage, :target_coverage)
```



```
5  coverage_strategy = [  
6    CoverageTarget.new(  
7      'order_processor.rb',  
8      :high,  
9      12,  
10     true,  
11     0,  
12     80 # PRIORITY 1: High change, high bugs, planned work  
13   ),  
14   CoverageTarget.new(  
15     'payment_gateway.rb',  
16     :medium,  
17     5,  
18     true,  
19     20,  
20     70 # PRIORITY 2: Planned changes  
21   ),  
22   CoverageTarget.new(  
23     'legacy_reporting.rb',  
24     :low,  
25     2,  
26     false,  
27     0,  
28     0 # PRIORITY: None - stable, no planned changes  
29   )  
30 ]  
31  
32 # Focus testing effort on code that will change  
33 # Don't test stable legacy code that works and won't be touched
```

Coverage prioritization strategy.

Test Organization

Legacy test suites need structure that standard test organization doesn't provide. You have characterization tests (documenting existing behavior), golden masters (complex output snapshots), regression tests (bug fixes), and unit tests for newly extracted code. Mixing them together creates confusion about what each test means and when it should change.



Separate tests by purpose. Characterization tests should be clearly marked – these document what the system *does*, not what it *should do*, and changing them requires investigation. Regression tests should reference bug tracker IDs so future developers understand the history. Unit tests for extracted code follow normal conventions.

Naming matters too. A prefix like `[CHAR]` for characterization tests or `[BUG-1234]` for regressions makes intent scannable. When a test fails, the name tells you whether you broke something or whether you're seeing expected change from intentional refactoring.

test_organization_spec.rb

```
1  # Organizing legacy code tests
2
3  # Directory structure
4  # spec/
5  # |— characterization/          # Capture existing behavior
6  # |   |— order_processing_spec.rb
7  # |   |— payment_gateway_spec.rb
8  # |— golden_masters/           # Complex output snapshots
9  # |   |— invoice_generation/
10 # |   |— report_generation/
11 # |— integration/              # Cross-component tests
12 # |   |— order_to_invoice_spec.rb
13 # |— unit/                     # New extracted code
14 # |   |— pricing_service_spec.rb
15 # |   |— validation_service_spec.rb
16
17 # Test naming for legacy code
18 RSpec.describe OrderProcessor do
19   context 'characterization - existing behavior' do
20     # These tests document what the code DOES
21     # Don't change them unless behavior intentionally changes
22
23     it '[CHAR] returns order total including tax for CA customers' do
24       # [CHAR] prefix indicates characterization test
25     end
26
27     it '[CHAR] handles null shipping address by using billing' do
28       # Documents discovered behavior (maybe unexpected)
29     end
30   end
31 end
```

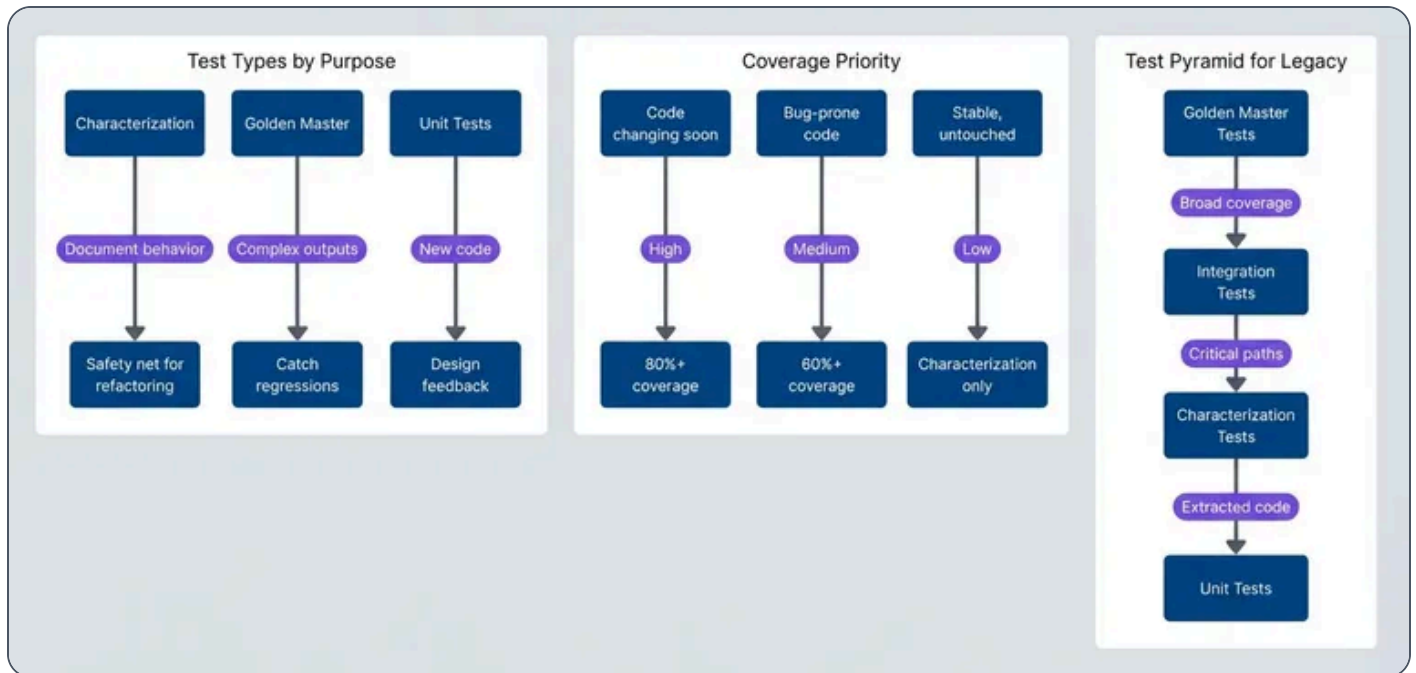


```

32 context 'unit - extracted logic' do
33   # These test new, extracted, well-designed code
34
35   it 'calculates discount for premium customers' do
36     # Standard unit test for extracted PricingService
37   end
38 end
39
40 context 'regression - bug fixes' do
41   # Tests added when bugs are fixed
42
43   it '[BUG-1234] prevents negative quantities' do
44     # Reference to bug tracker
45   end
46 end
47 end

```

Test organization for legacy code.



Legacy testing strategy.



Note that this inverts the traditional test pyramid. In greenfield development, unit tests form the base. In legacy code, you start with broad characterization and golden master tests at the top – they’re faster to write and provide immediate safety. As you extract and refactor, unit tests grow from the bottom up.

WARNING

Don’t aim for 100% coverage on legacy code. Aim for coverage where you need confidence: code you’re changing, code with bug history, and critical paths. Testing stable legacy code that works is wasteful.

Conclusion

Legacy code isn’t a curse – it’s working software that’s earned its complexity through years of real-world use. The techniques in this article let you approach it systematically rather than fearfully.

Start with characterization tests. Run the code, observe what happens, write it down. Don’t judge whether the behavior is correct – just document it. These tests become your safety net, catching unintended changes during refactoring.

Find seams where you can alter behavior without modifying code. Object seams let you substitute implementations. Link seams let you intercept at module boundaries. Preprocessor seams let you swap behavior based on environment. Every language has them; you just need to recognize them.

Break dependencies incrementally. Extract and Override gets you started with minimal change. Parameterize Constructor creates explicit dependency graphs. Instance Delegator handles static cling. Each technique trades design improvement against change risk – choose based on your context.



✓ SUCCESS

The key insight: you don't need to understand legacy code to test it. Characterization tests capture behavior you can observe. Seams let you isolate without understanding. The myth of "untestable" code usually means "code that's hard to test with conventional techniques." With these patterns, almost any code becomes testable – the question is whether the investment is worth it for code that may never change.

Use the Strangler Fig pattern for larger extractions. Wrap legacy in a facade, extract one responsibility at a time into tested components, gradually hollow out the legacy code until it's gone. Every step is deployable. No big-bang rewrites.

Prioritize ruthlessly. Test code that's changing, code that's breaking, code that's critical. Leave stable legacy code alone – characterization tests catch accidental changes, but comprehensive testing of code that won't change is waste.

Copyright © 2023 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/legacy-code-testing-characterization-tests-seams>

